

ONLINE DONATION MANAGEMENT SYSTEM



DBMS Course Project

Course code: CS403PC

Academic Year: 2025-26

Submitted by

M.Varalaxmi

246Y1A66B0

Department of CSE (AI&ML)

Project Guide

Mr. Dandugudum Mahesh

Assistant Professor

Department of CSE (AI&ML)



SUMATHI REDDY INSTITUTE OF TECHNOLOGY FOR WOMEN

(Autonomous Institution – UGC, Govt. of India)

(Affiliated to JNTUH, Hyderabad, Approved by AICTE - Accredited by NBA & NAAC – 'A' Grade)

Ananthasagar, Hasanparthy mandal, Warangal, Telangana 506371

INDEX

S.NO	TITLE	Page No's
1	Introduction	3
2	Database Design	4
	2.1 ER Diagram	4
	2.1.1 Online Donation Management Systems Features	4
	2.1.2 Entities and Attributes for Online Donation Management Systems	5
	2.1.3 Relationships between the Entities	5
	2.1.4 Representation of ER Diagram	6
	2.2 Tables and Relationships	7
3	Implementation	
	3.1 Table Creation	9
	3.2 Data Insertion	13
	3.3 SQL Queries	15
	3.4 Advanced SQL Features	17
4	Results	19
5	Conclusion	20

1.Introduction

The Online Donation Management System is a modern database application designed to facilitate and manage donation activities efficiently through digital platforms. In today's world, where technology plays a vital role in everyday transactions, online donation systems have become an essential tool for supporting charitable organisations, non-governmental organisations (NGOs), and social causes.

The primary purpose of this system is to provide a centralised platform where donors can contribute funds easily, and organisations can manage, track, and utilise these donations effectively. It eliminates the need for manual record-keeping and reduces errors by storing all data in a structured and organised database.

This system maintains detailed information about donors, organisations, donations, and payment transactions. It ensures transparency and accountability by allowing users to view donation history and track how funds are distributed. Additionally, it supports multiple organisations, enabling donors to contribute to different causes based on their interests.

The Online Donation Management System is developed using a relational database management system such as MySQL. It demonstrates important DBMS concepts, including data modelling, normalization, entity-relationship design, and implementation of SQL queries. These concepts help in maintaining data integrity, reducing redundancy, and improving system performance.

Furthermore, the system can be enhanced by incorporating advanced database features such as views, stored procedures, and triggers to automate operations and improve efficiency. It can also be integrated with web or mobile applications to provide a user-friendly interface for real-time donation management.

In conclusion, this project provides a practical understanding of how database systems are used in real-world applications and highlights the importance of efficient data management in online donation platforms.

2. Database Design

The database design is one of the most important parts of the Online Donation Management System. It defines how data is structured, stored, and connected within the system. A well- designed database ensures data consistency, avoids redundancy, and improves performance. In this project, the database is designed using the Entity-Relationship (ER) model, which helps in visualising the structure of the database and the relationships between different entities.

2.1 ER Diagram

Online Donation Management Systems allow a wide range of devices, sensors, and functionalities which focus at enhancing the comfort, security, and energy efficiency of homes. These systems allow users to remotely monitor and control various aspects of their homes, such as lighting, heating, security cameras, and more, using smartphones or voice commands.

By defining entities such as users, devices, rooms, sensors, and schedules, these systems can automate routine tasks, adapt to user preferences, and provide valuable insights into energy usage and home security.

2.1.1 Online Donation Management Systems Features:

- **Donor Registration and Login:** The donors create accounts and securely log in to make donations.
- **Online Donation Facility:** Donors can contribute money to different organizations through online payment methods.
- **Multiple Organization Support:** The system allows multiple NGOs or organizations to register and receive donations.
- **Donation Tracking:** All donation transactions are recorded and can be tracked easily.
- **Secure Payment Management:** Payment details such as method and status are stored securely.
- **Transparency and Accountability:** Donors can view their donation history, ensuring trust and transparency.

2.1.2 Entities and Attributes for online Donation Management Systems

Let's Defining Entities for Online Donation Management Systems are:

1. **Donors:** Donors are individuals who donates to desired organisations.

- **donor_id (Primary Key):** Unique ID for each donor.

- name: name of the donor.
- email: Email address.
- phone: contact number

2. Organisation: These organization can be private / government.

- org_id(Primary key): Unique ID for each organisation
- name: Organisation name
- type: NGO / Trust / Charity
- address: Location of organisation.

3. Donation: The donors donates the amount to the organization.

- donation_id(Primary key): Unique ID for each donation.
- amount: Donation amount.
- date: Date of donation.
- donor_id(Foreign Key): Reference to donor.
- org_id(Foreign Key): Reference to organization.

4. Payment:

- payment_id(Primary Key): Unique ID for each payment.
- method: Payment method(UPI, Card, Net banking)
- status: Payment status (Completed/Pending)
- donation_id(Foreign Key): Reference to donation.

2.1.3 Relationship between these Entities

1. Donor – Donation Relationship

- One donor can make multiple donations
- Each donation belongs to one donor
- Relationship Type: One-to-Many (1:M)

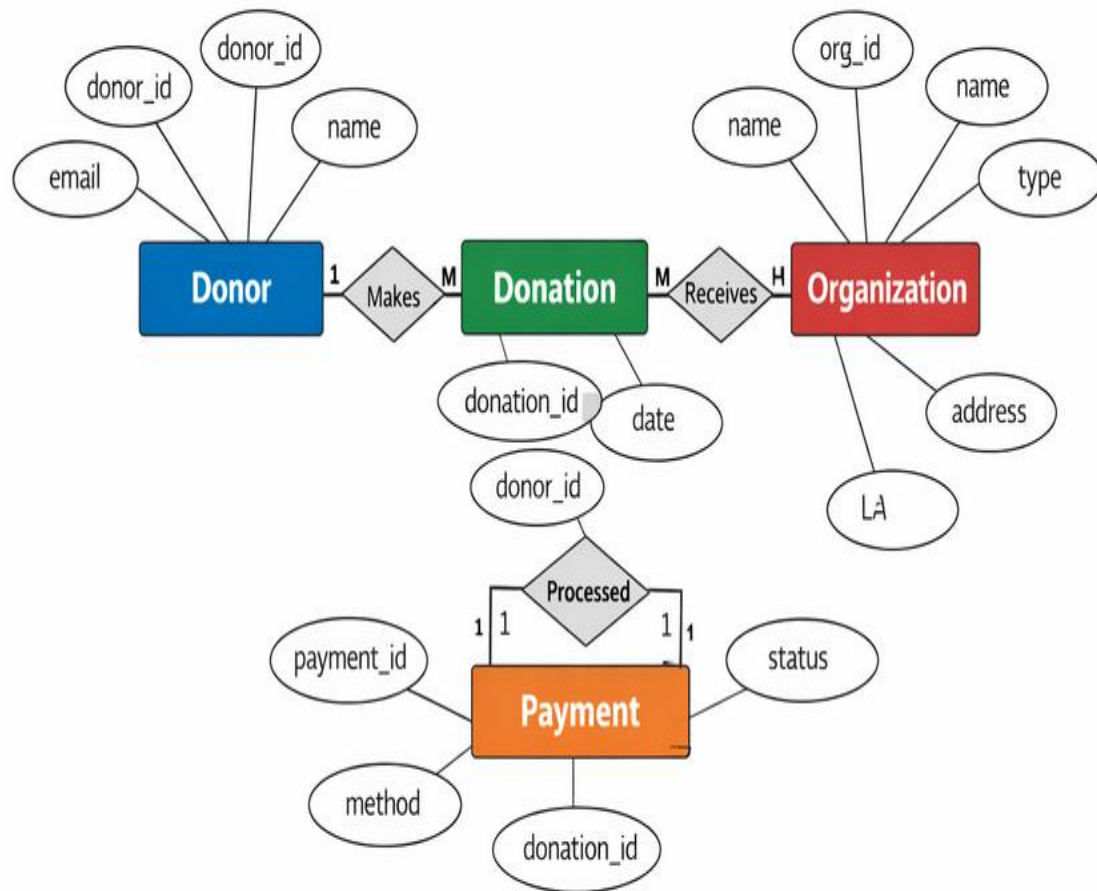
2. Organisation – Donation Relationship

- One organization can receive many donations
- Each donation is given to one organization
- Relationship Type: One-to-Many (1:M)

3. Donation – Payment Relationship

- Each donation has one payment record
- Each payment is linked to one donation
- Relationship Type: One-to-One (1:1)

2.1.4 Representation of ER Diagram



2.2 Tables and Relationships:

Based on the ER Diagram, the database is implemented using relational tables. Each entity is converted into a table, and relationships are maintained using primary and foreign keys.

1. Donor Table

The Donor table stores information about the donors.

Fields:

- donor_id : Primary Key
- name: name of the donor.
- email: Email address.
- phone: contact number

Explanation:

This table is the base of the system. Each donor is uniquely identified by donor_id.

2. Organisation Table

The organisation table stores information about the organisation.

Fields:

- org_id: Primary key
- name: Organisation name
- type: NGO / Trust / Charity
- address: Location of organisation.

Explanation:

This table consists of organisation details. The donors would be able to see these details.

3. Donation Table

The Donation table stores information about the amount donated by the donor to the particular organisation.

Fields:

- donation_id: Primary key
- amount: Donation amount
- date: Date of donation
- donor_id: Foreign Key

- org_id: Foreign Key

Explanation:

This table is linked to the Donor table through donor_id. It shows which donation is made by the donor.

The table is also linked to the Organisation table through org_id. It shows the donation made by donor to the particular organization.

4. Payment Table

The Payment table stores the information about transaction details.

Fields:

- payment_id: Primary Key
- method: Payment method(UPI, Card, Net banking)
- status: Payment status (Completed/Pending)
- donation_id: Foreign Key

Explanation:

This table is linked to the Donation table through donation_id. It shows the payment details of the donation made by the donor.

Relationships Between Tables

The relationships between tables are implemented using foreign keys:

- Donor → Donation
Donation, donor_id references Donor, donor_id
- Organisation → Donation
Donation, org_id references Organisation, org_id
- Donation → Payment
Payment, donation_id references Donation, donation_id

3.Implementation

3.1 Table Creation

Create a database

```
mysql> create database project;  
Query OK, 1 row affected (0.03 sec)
```

```
mysql> use project;  
Database changed
```

1.Donor Table:

Description: Stores the information of the donors.

Attributes:

- donor_id (Primary Key)
- name
- email
- phone

SQL:

```
CREATE TABLE donor(  
    donor_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(50),  
    phone VARCHAR(50)  
);
```

```
mysql> CREATE TABLE donor(donor_id INT PRIMARY KEY,name VARCHAR(50),email VARCHAR(50),phone VARCHAR(15));  
Query OK, 0 rows affected (0.18 sec)
```

```
mysql> desc donor;
```

Field	Type	Null	Key	Default	Extra
donor_id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
email	varchar(50)	YES		NULL	
phone	varchar(15)	YES		NULL	

4 rows in set (0.06 sec)

2. Organisation Table:

Description: Stores the information of the organisations.

Attributes:

- org_id (Primary Key)
- name
- type
- address

SQL:

```
CREATE TABLE organisation (  
    org_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    type VARCHAR(50),  
    address VARCHAR(100)  
);
```

```
mysql> CREATE TABLE organisation(org_id INT PRIMARY KEY,name VARCHAR(50),  
-> type VARCHAR(50),address VARCHAR(100));  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> desc organisation;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| org_id | int           | NO   | PRI | NULL    |       |  
| name   | varchar(50)   | YES  |     | NULL    |       |  
| type   | varchar(50)   | YES  |     | NULL    |       |  
| address | varchar(100) | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.01 sec)
```

3. Donation Table:

Description: Stores the information of the donations made by the donors.

Attributes:

- Donation_id (PRIMARY KEY)
- Amount
- Date
- Donor_id (FOREIGN KEY)

- Org_id (FOREIGN KEY)

SQL:

```
CREATE TABLE donation(
    donation_id INT PRIMARY KEY,
    amount DECIMAL(10,2),
    date DATE,
    donor_id INT,
    org_id INT,
    FOREIGN KEY (donor_id) REFERENCES donor(donor_id),
    FOREIGN KEY (org_id) REFERENCES organisation(org_id)
);
```

```
mysql> CREATE TABLE donation(donation_id INT PRIMARY KEY,amount DECIMAL(10,2
),date DATE,donor_id INT,org_id INT,FOREIGN KEY (donor_id) REFERENCES donor(
donor_id),FOREIGN KEY (org_id) REFERENCES organisation(org_id));
Query OK, 0 rows affected (0.12 sec)
```

```
mysql> desc donation;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| donation_id | int           | NO   | PRI | NULL    |       |
| amount      | decimal(10,2) | YES  |     | NULL    |       |
| date        | date          | YES  |     | NULL    |       |
| donor_id    | int           | YES  | MUL | NULL    |       |
| org_id      | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.03 sec)
```

4. Payment Table:

Description: This table stores the details of the transactions.

Attributes:

- payment_id (PRIMARY KEY)
- method
- status
- donation_id (FOREIGN KEY)

SQL:

```

CREATE TABLE payment(
    payment_id INT PRIMARY KEY,
    method VARCHAR(20),
    status VARCHAR(20),
    donation_id INT,
    FOREIGN KEY (donation_id) REFERENCES donation(donation_id)
);

```

```

mysql> CREATE TABLE payment(payment_id INT PRIMARY KEY,method VARCHAR(20),st
atus VARCHAR(20),donation_id INT,FOREIGN KEY (donation_id) REFERENCES donati
on(donation_id));
Query OK, 0 rows affected (0.05 sec)

```

```

mysql> desc payment;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| payment_id | int           | NO   | PRI | NULL    |       |
| method      | varchar(20)   | YES  |     | NULL    |       |
| status      | varchar(20)   | YES  |     | NULL    |       |
| donation_id | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

3.2 Data Insertion

1. Donor Table

```
mysql> INSERT INTO donor VALUES
-> (1,'Ravi','ravi@gmail.com','9876543210'),
-> (2,'Swathi','swathi@gmail.com','7208886971'),
-> (3,'Kavi','kavi@gmail.com','6308336317');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> select * from donor;
+-----+-----+-----+-----+
| donor_id | name   | email           | phone   |
+-----+-----+-----+-----+
| 1        | Ravi   | ravi@gmail.com  | 9876543210 |
| 2        | Swathi | swathi@gmail.com | 7208886971 |
| 3        | Kavi   | kavi@gmail.com  | 6308336317 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. Organisation Table

```
mysql> INSERT INTO organisation VALUES
-> (101,'Helping Hands','NGO','Hyderabad'),
-> (102,'Orphanage','Public','Warangal');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> select * from organisation;
+-----+-----+-----+-----+
| org_id | name           | type   | address   |
+-----+-----+-----+-----+
| 101    | Helping Hands | NGO    | Hyderabad |
| 102    | Orphanage     | Public | Warangal  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

3. Donation Table

```
mysql> INSERT INTO donation VALUES
-> (1001,5000,'2026-04-01',1,101),
-> (1002,10000,'2026-05-05',2,102),
-> (1003,1000,'2026-05-05',2,101);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> select * from donation;
```

donation_id	amount	date	donor_id	org_id
1001	5000.00	2026-04-01	1	101
1002	10000.00	2026-05-05	2	102
1003	1000.00	2026-05-05	2	101

```
3 rows in set (0.00 sec)
```

4. Payment Table

```
mysql> INSERT INTO payment VALUES
-> (501, 'UPI', 'Completed', 1001),
-> (502, 'Card', 'Completed', 1002),
-> (503, 'UPI', 'Completed', 1003);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> select * from payment;
```

payment_id	method	status	donation_id
501	UPI	Completed	1001
502	Card	Completed	1002
503	UPI	Completed	1003

```
3 rows in set (0.00 sec)
```

3.3 SQL Queries

1. List all of the donors.

Returns the donors table.

```
mysql> SELECT * FROM donor;
+-----+-----+-----+-----+
| donor_id | name  | email                | phone  |
+-----+-----+-----+-----+
| 1        | Ravi  | ravi@gmail.com       | 9876543210 |
| 2        | Swathi | swathi@gmail.com     | 7208886971 |
| 3        | Kavi  | kavi@gmail.com       | 6308336317 |
+-----+-----+-----+-----+
3 rows in set (0.05 sec)
```

2. Find the total amount of donations.

Returns the sum of donations.

```
mysql> SELECT SUM(amount) FROM donation;
+-----+
| SUM(amount) |
+-----+
| 16000.00    |
+-----+
1 row in set (0.01 sec)
```

3. Find donations with donor names.

Returns the donor names and donations made by them.

```
mysql> SELECT d.name, dn.amount
-> FROM donor d
-> JOIN donation dn ON d.donor_id = dn.donor_id;
+-----+-----+
| name  | amount |
+-----+-----+
| Ravi  | 5000.00 |
| Swathi | 10000.00 |
| Swathi | 1000.00 |
+-----+-----+
3 rows in set (0.06 sec)
```

4. Count donations per organization.

Return the donations donated to the organisations.

```
mysql> SELECT org_id, COUNT(*) FROM donation GROUP BY org_id;
```

org_id	COUNT(*)
101	2
102	1

```
2 rows in set (0.01 sec)
```

5. Find all donations made by a specific donor.

```
mysql> SELECT *
-> FROM donation
-> WHERE donor_id = 1;
```

donation_id	amount	date	donor_id	org_id
1001	5000.00	2026-04-01	1	101

```
1 row in set (0.01 sec)
```

6.

6. Find donations made on a specific date.

```
mysql> SELECT *
-> FROM donation
-> WHERE date = '2026-04-01';
```

donation_id	amount	date	donor_id	org_id
1001	5000.00	2026-04-01	1	101

```
1 row in set (0.00 sec)
```

7. Show organisation name with total donations received.

```
mysql> SELECT o.name, SUM(dn.amount) AS total_donations
-> FROM organisation o
-> JOIN donation dn ON o.org_id = dn.org_id
-> GROUP BY o.name;
```

name	total_donations
Helping Hands	6000.00
Orphanage	10000.00

```
2 rows in set (0.01 sec)
```

8. Find highest donation amount.


```
mysql> SELECT MAX(amount) AS highest_donation
-> FROM donation;
+-----+
| highest_donation |
+-----+
|          10000.00 |
+-----+
1 row in set (0.01 sec)
```

9. Find lowest donation amount.

```
mysql> SELECT MIN(amount) AS lowest_donation
-> FROM donation;
+-----+
| lowest_donation |
+-----+
|          1000.00 |
+-----+
1 row in set (0.03 sec)
```

10. Show payment details with donation amount.

```
mysql> SELECT p.method, p.status, d.amount
-> FROM payment p
-> JOIN donation d ON p.donation_id = d.donation_id;
+-----+-----+-----+
| method | status | amount |
+-----+-----+-----+
| UPI    | Completed | 5000.00 |
| Card   | Completed | 10000.00 |
| UPI    | Completed | 1000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

3.4 Advanced SQL Features

1. Create a VIEW to display name and donation amount.

```
mysql> CREATE VIEW donor_donation AS
-> SELECT d.name, dn.amount
-> FROM donor d
-> JOIN donation dn ON d.donor_id = dn.donor_id;
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> SELECT * FROM donor_donation;
```

name	amount
Ravi	5000.00
Swathi	10000.00
Swathi	1000.00

```
3 rows in set (0.01 sec)
```

2. Create a procedure to get GetDonations.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetDonations()
-> BEGIN
-> SELECT * FROM donation;
-> END //
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> CALL GetDonations();
```

donation_id	amount	date	donor_id	org_id
1001	5000.00	2026-04-01	1	101
1002	10000.00	2026-05-05	2	102
1003	1000.00	2026-05-05	2	101

```
3 rows in set (0.00 sec)
```

3. Create a trigger before insert.

```
mysql> CREATE TRIGGER check_amount
-> BEFORE INSERT ON donation
-> FOR EACH ROW
-> SET NEW.amount = IF(NEW.amount < 0,0, New.amount);
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO donation VALUES
-> (2001,-5000, '2026-07-01',1,101);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM donation;
```

donation_id	amount	date	donor_id	org_id
1001	5000.00	2026-04-01	1	101
1002	10000.00	2026-05-05	2	102
1003	1000.00	2026-05-05	2	101
2001	0.00	2026-07-01	1	101

```
4 rows in set (0.00 sec)
```

4. Results and Discussion

The Online Donation Management System database was successfully designed and implemented using MySQL. All the required tables such as Donor, Donations, Organisations, and Payment were created with proper primary and foreign key constraints.

Sample data was inserted into all tables, and the relationships between entities were maintained correctly. The database operations were tested using various SQL queries including joins, aggregation functions, nested queries, and set operations. All queries produced accurate and expected results.

The implementation of advanced SQL features such as views, stored procedures, triggers was also successful. The stored procedures returned correct outputs based on input parameters, triggers worked automatically during data insertion, and views simplified data retrieval.

Normalization techniques were applied up to Third Normal Form (3NF) and BCNF, which helped in reducing redundancy and improving data consistency.

Key Observations:

- Data was stored efficiently without duplication
- Queries executed successfully with correct outputs
- Relationships between tables were properly maintained
- Automation features like triggers and schedules worked effectively

5. Conclusion:

The Online Donation Management System project was successfully designed and implemented using MySQL, demonstrating how donors can make donations to various organisations.

Throughout the project, key DBMS concepts such as ER modeling, relational schema design, and normalisation were applied. The database was normalized up to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF), ensuring minimal redundancy and improved data integrity. Proper use of primary and foreign keys helped maintain strong relationships between tables.

Various SQL operations, including basic queries, joins, aggregation, nested queries, and set operations, were implemented successfully to retrieve meaningful information from the database. Advanced features such as views, stored procedures, triggers, and cursors were also incorporated, enhancing the functionality and automation of the system.

The project provided valuable hands-on experience in designing and implementing a database system. It improved understanding of how databases are used in real-life to make donations can be done easily.

Furthur Enhancements:

- We can implement web - interface to make donations easy.
- We can implement payment gateway integration.
- We can add admin interface.
- It would be very good to integrate it as a mobile application.

In conclusion, this project successfully demonstrates how a well-designed database system can efficiently manage donations made by different donors to different organisations.